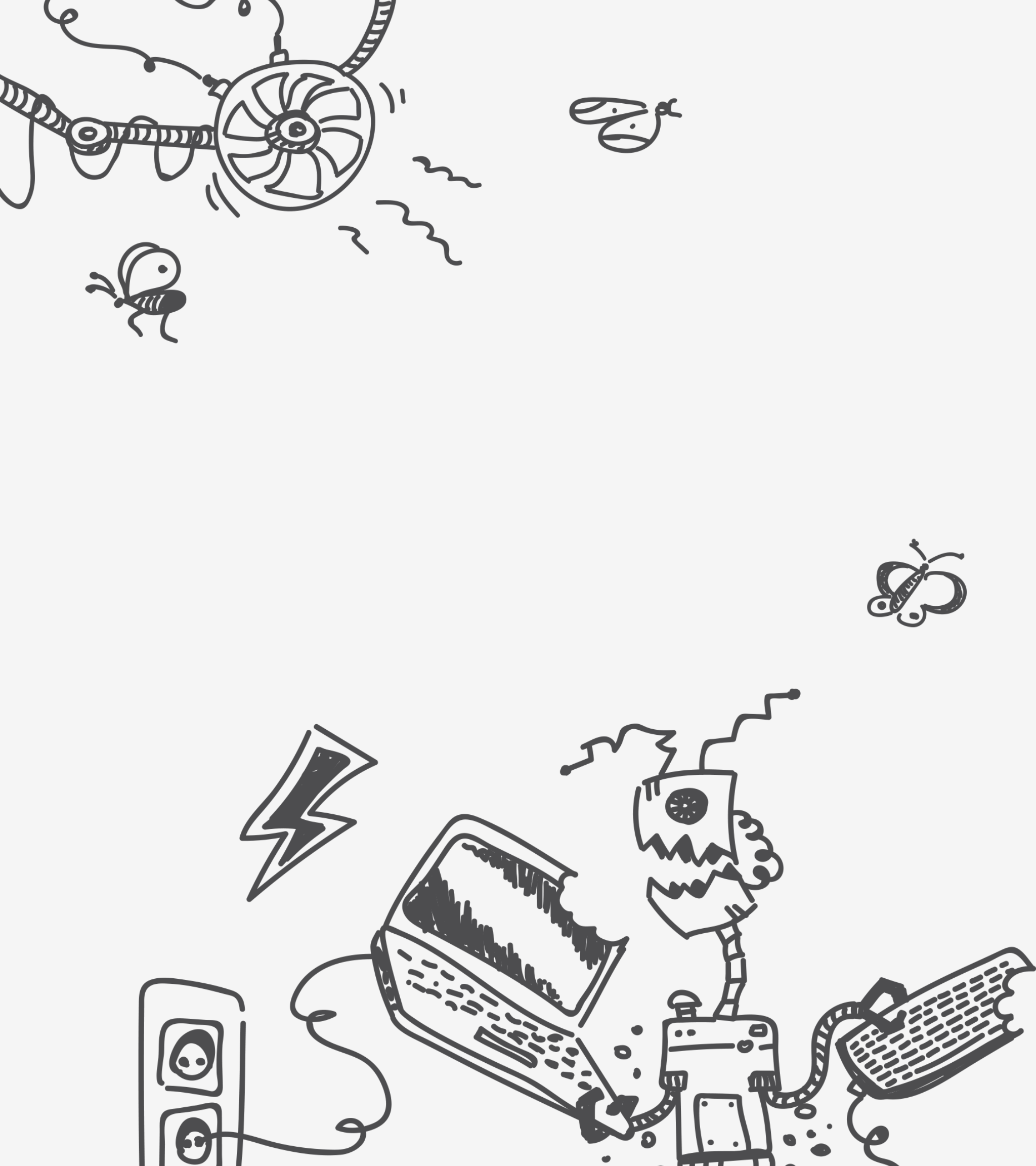
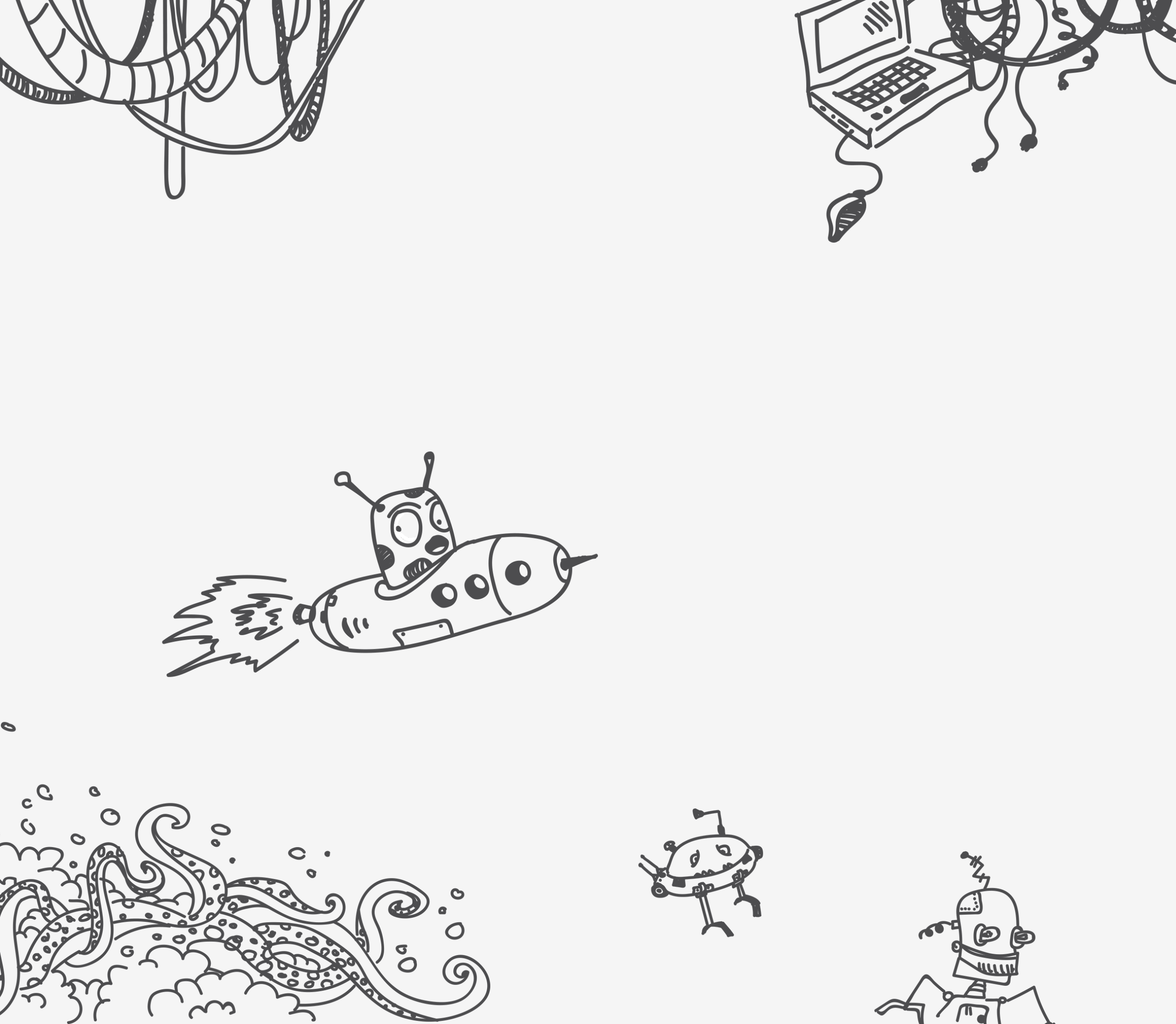




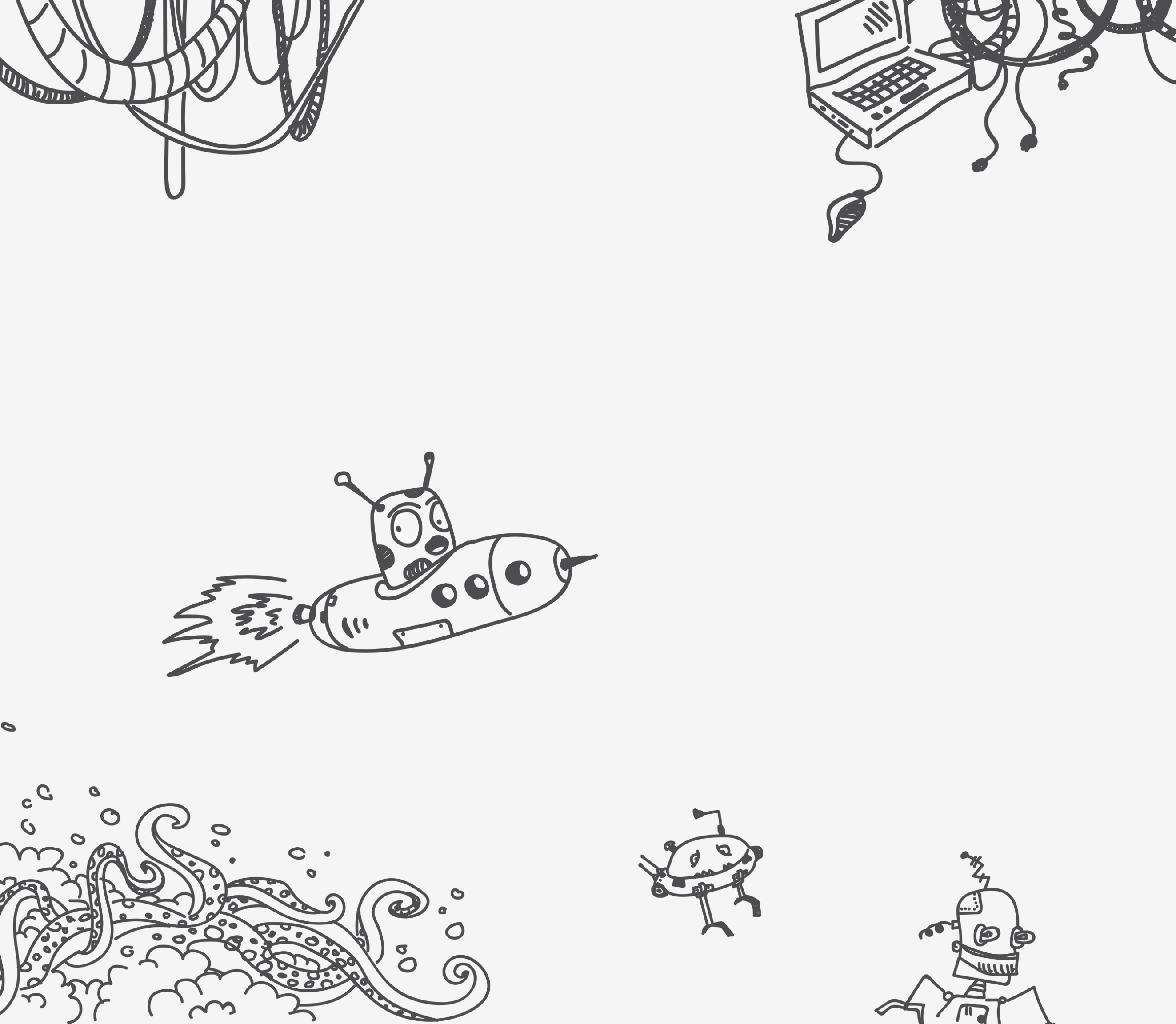
Metrics of regression models - description and example of calculations in Python



Introduction

Introduction

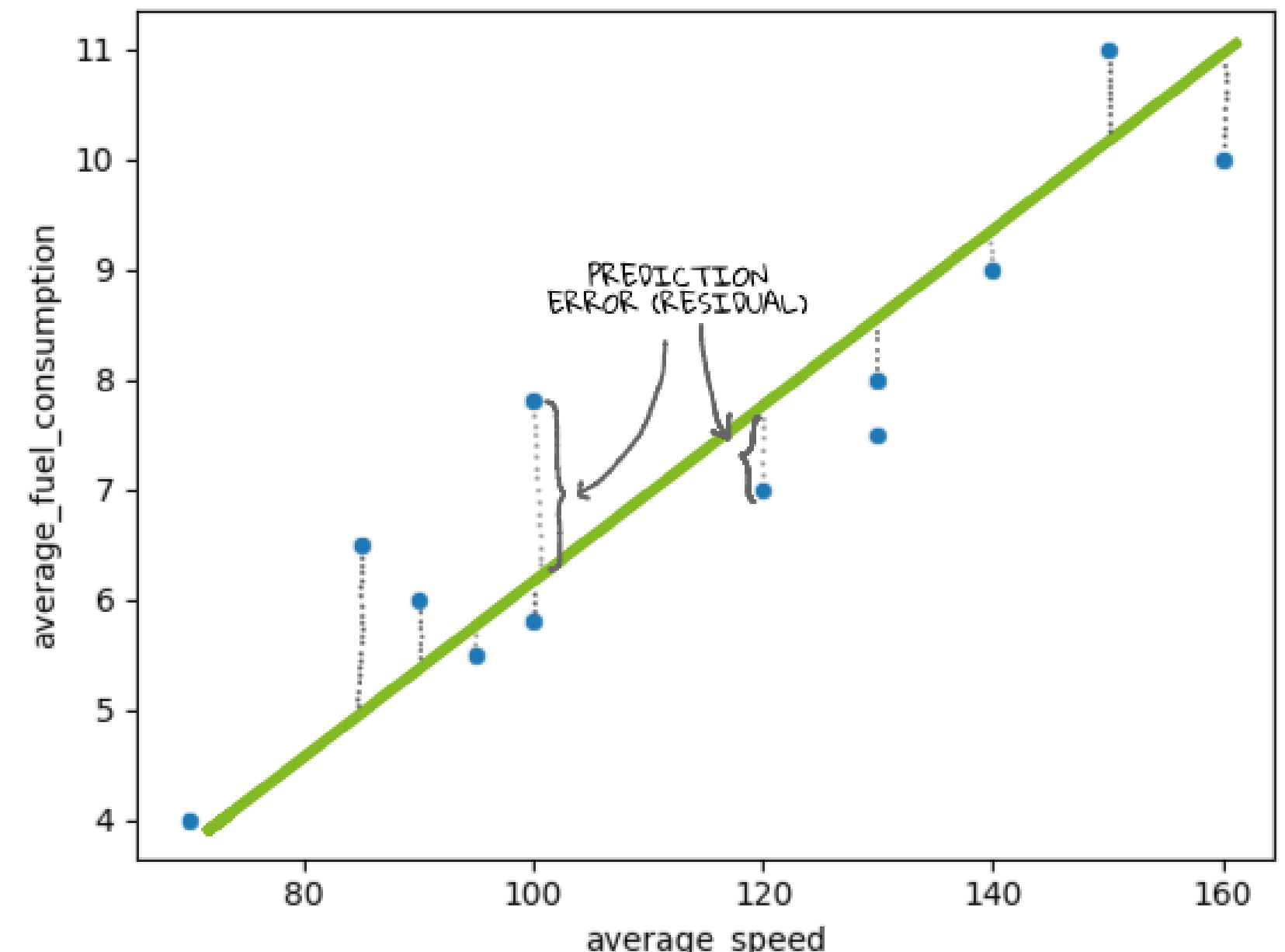
Regression is one of the problems solved in supervised learning. It involves finding a relationship between the independent variables and the dependent variable in order to obtain a result in the form of, for example, the price of an object (e.g., real estate), the amount of wages, or sales results of products in a store. It can be said that regression answers the question: *how much?* When we build a machine learning model, we want to create the best solution possible. To decide whether our model is good or not, we need a verification tool. In machine learning, such a tool is metrics. In this chapter, we will touch on the five most popular metrics used to evaluate regression models.



Definition of error of the regression

Definition of error of the regression

In regression problems, prediction error (i.e., the error of our model) is used to determine model performance. Prediction error is the same as residual values and is defined as the difference between actual and predicted values.



The general formula for *residual value* is:

residual value = *actual value* - *value predicted by the model*

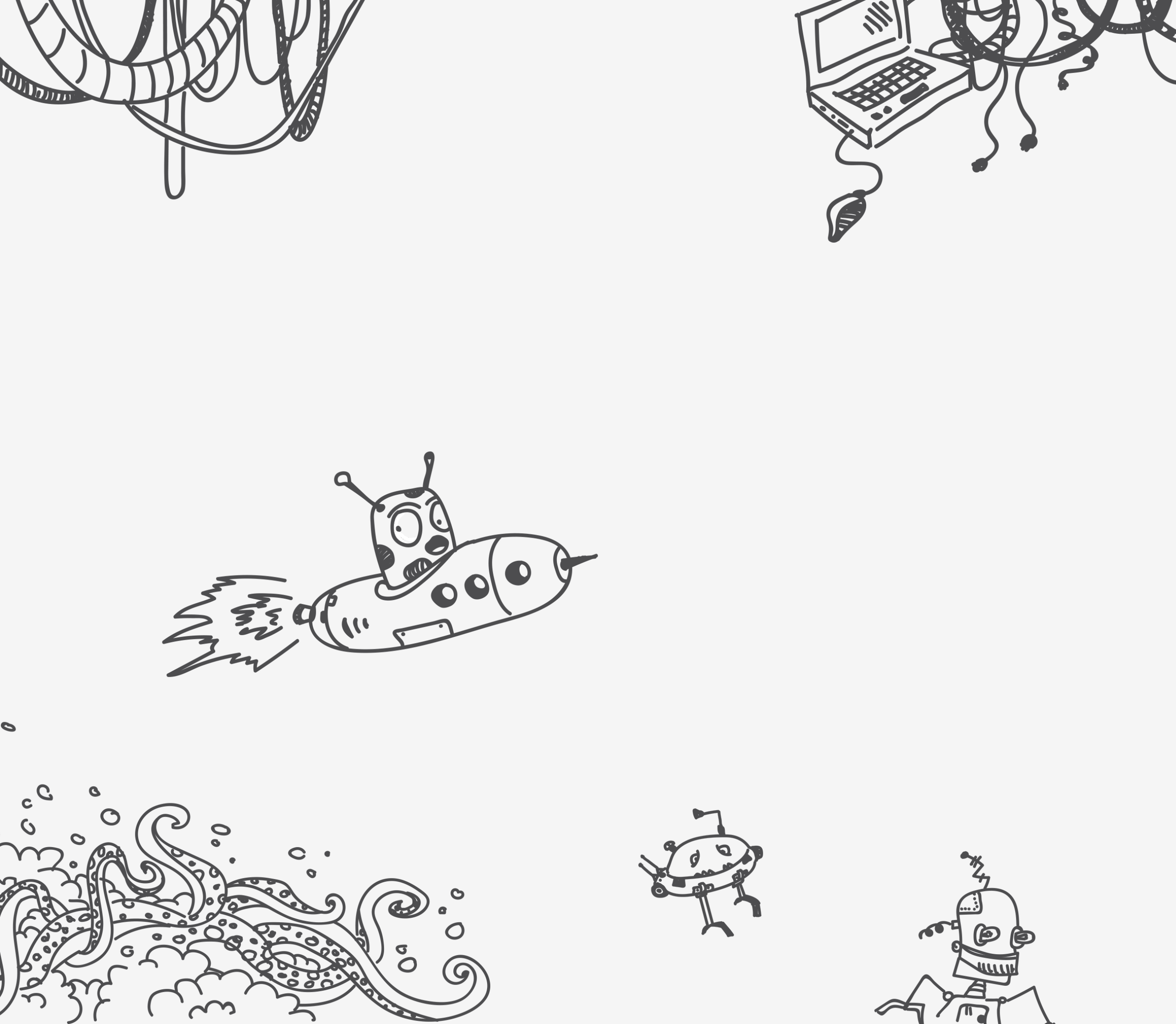
The image shows how a linear regression model tries to fit a straight line that gives the smallest difference between predicted and actual values. This difference, or residuals, are very important in determining the quality of the model.

Definition of error of the regression

Prediction errors can be tested in various ways:

- All residuals are 0, which means that the model is perfect (i.e. probably overtrained). The further the residual values are from zero, the less accurate the model.
- The average prediction error is different from zero, which may indicate that the model is systematically biased (underfit or overfit).
- Residuals form patterns, and this may indicate that the model is ineffective in interpreting some properties in the data.

In fact, models predict values on huge volumes of data, so an accurate analysis of values (one by one) is not possible. Nevertheless, we can visualize them in graphs. In addition to this, summary measures have been introduced, which take all the residual values and condense them into a single numerical value (also called: **metric**) that represents the effectiveness of our model.



Mean Absolute Error

Mean Absolute Error

Mean absolute error involves calculating the average of the absolute differences between the actual and predicted values in a data set.

We can describe the definition of mean absolute error with the formula:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

where:

- $\sum_{i=1}^n$ - is a sum of values from i to n . E.g. for: $i = 1$ i $n = 5$ the equation is a sum of:
 $|y_1 - \hat{y}_1| + |y_2 - \hat{y}_2| + \dots + |y_5 - \hat{y}_5|$
- $|\dots|$ - denotes the absolute value, which is used to avoid negative error values,
- n - the number of observations in the data set,
- y_i - the actual value of the dependent variable for the i -th observation,
- $\hat{y}_i$ - the value predicted by the model for the i -th observation.

Mean Absolute Error

The absence of an absolute value would cause negative differences to cancel out positive differences. After summing all the differences (residues), we would get a non-representative result:

e.g. $\frac{(12-7)+(7-12)+(1-1)}{3} = 0$ would mean that the model is not wrong.

On the other hand, using the absolute value, we get: $\frac{(|12-7|+|7-12|+|1-1|)}{3} = \frac{10}{3} = 3.3333....$

If it turned out that **MAE** is 0, it could mean that our model is perfect, which would be inconsistent with the facts. A low **MAE** value means that the model is good. A large mean absolute error suggests that the model may have some trouble predicting values.

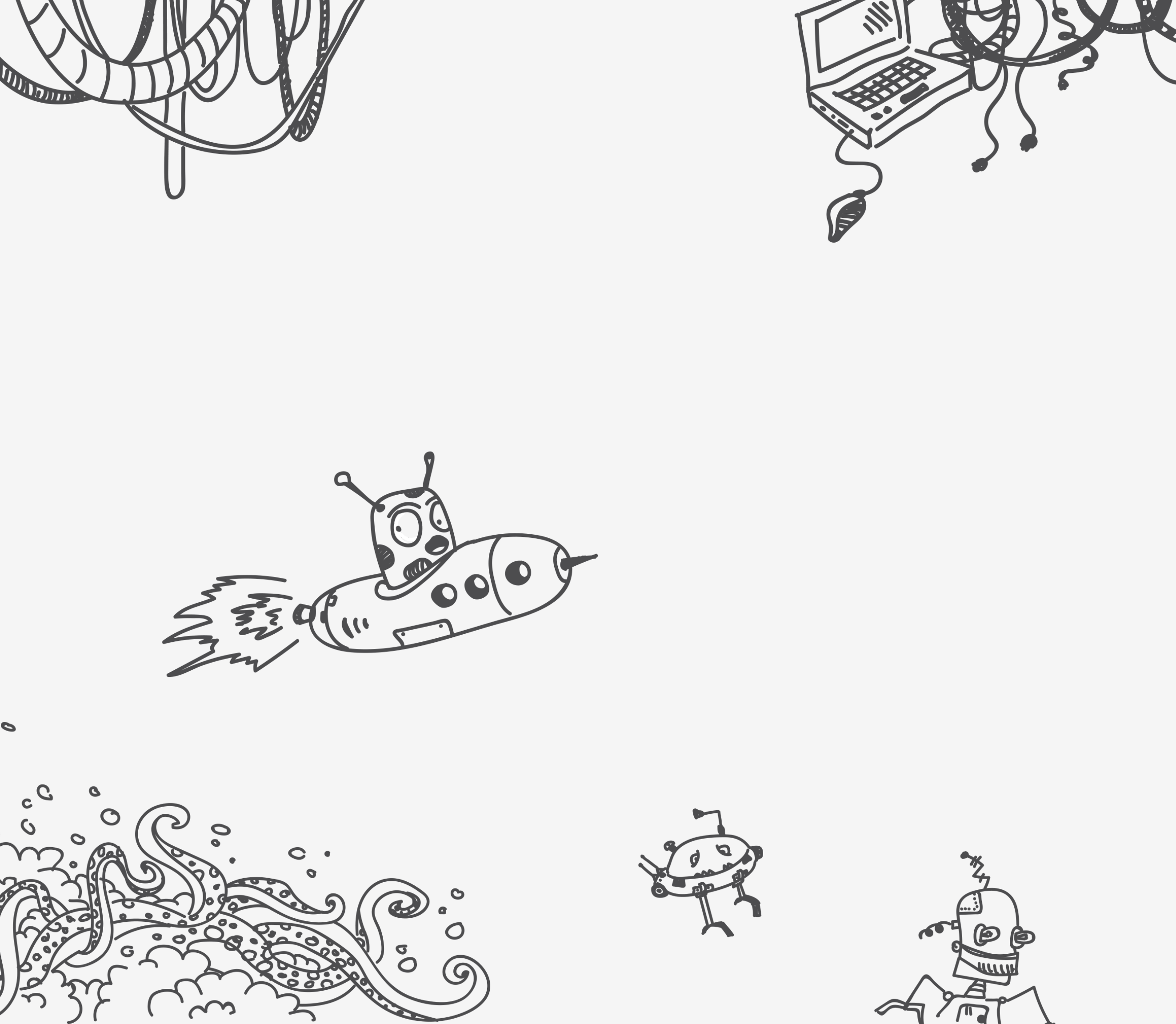
Mean Absolute Error

MAE is a metric that is easily interpretable because it is in the same units as the original data. This means that if the variable we are predicting is the price of a house and one of the actual values was 500,000 PLN, and our model predicted a price of 480,000 PLN then the absolute error was 20,000 PLN for that property.

Due to operating on absolute values, the MAE can handle outliers, which means that we have relatively objective information about the effectiveness of our model.

We can easily calculate the mean absolute error in Python by importing the *mean_absolute_error* method from the **Scikit-learn** library's **metrics** module:

```
from sklearn.metrics import mean_absolute_error
mae = mean_absolute_error(y_pred, y_true)
```



Mean Squared Error

Mean Squared Error

The mean squared error is based on calculating the average difference of squares between the actual values and those predicted by the model.

We can describe the definition of the mean square error with the formula:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- n - the number of observations in the data set,
- y_i - the actual value of the dependent variable for the i -th observation,
- $\hat{y}_i$ - the value predicted by the model for the i -th observation.
- $(\dots)^2$ - denotes the square of the error value

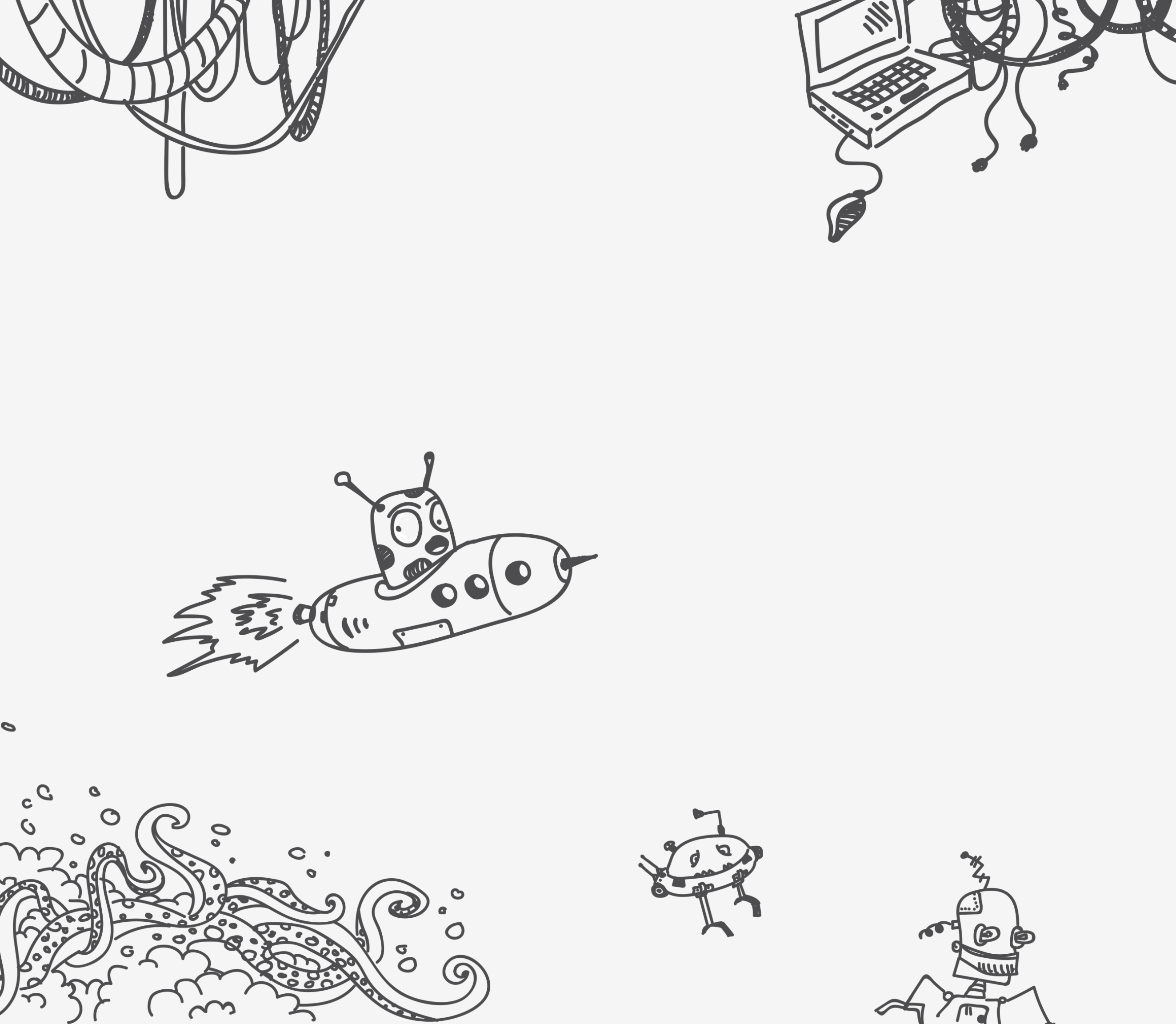
The way of calculating the MSE (squaring the difference between the actual values and those predicted by the model) makes the metric not immune to the occurrence of outliers. This way of calculation penalizes outliers the most, and thus the **MSE** value is increased by them. The root mean squared error uses a compounding operation to remove the sign of each error value and penalize large errors more.

Mean Squared Error

Suppose our model is generally great, but makes one very bad prediction. Raising the resulting error to the square will make the error worse and has a big impact on the value of the **MSE** metric. On the other hand, if all of the calculated errors are rather small, we can also get an unclear picture and miss the fact that realistically the model is not the best.

In the **Scikit-learn** library, we can easily calculate the mean squared error using the appropriate function:

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y_pred, y_true)
```



Root Mean Squared Error

Root Mean Squared Error

The root of the mean squared error is nothing more than the root of the previous metric, MSE. The rooting operation reduces the MSE to the units in which our prediction was made, making it much easier to interpret. Our goal is for the RMSE to be as low as possible - the lower the **RMSE**, the better the predictions of our model. The root of the mean squared error still has the same problem as the MSE - it doesn't deal well with outliers. In some cases, RMSE is useful when large model errors are undesirable (e.g., calculating creditworthiness).

The formula for RMSE is not very different from that for mean square error:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

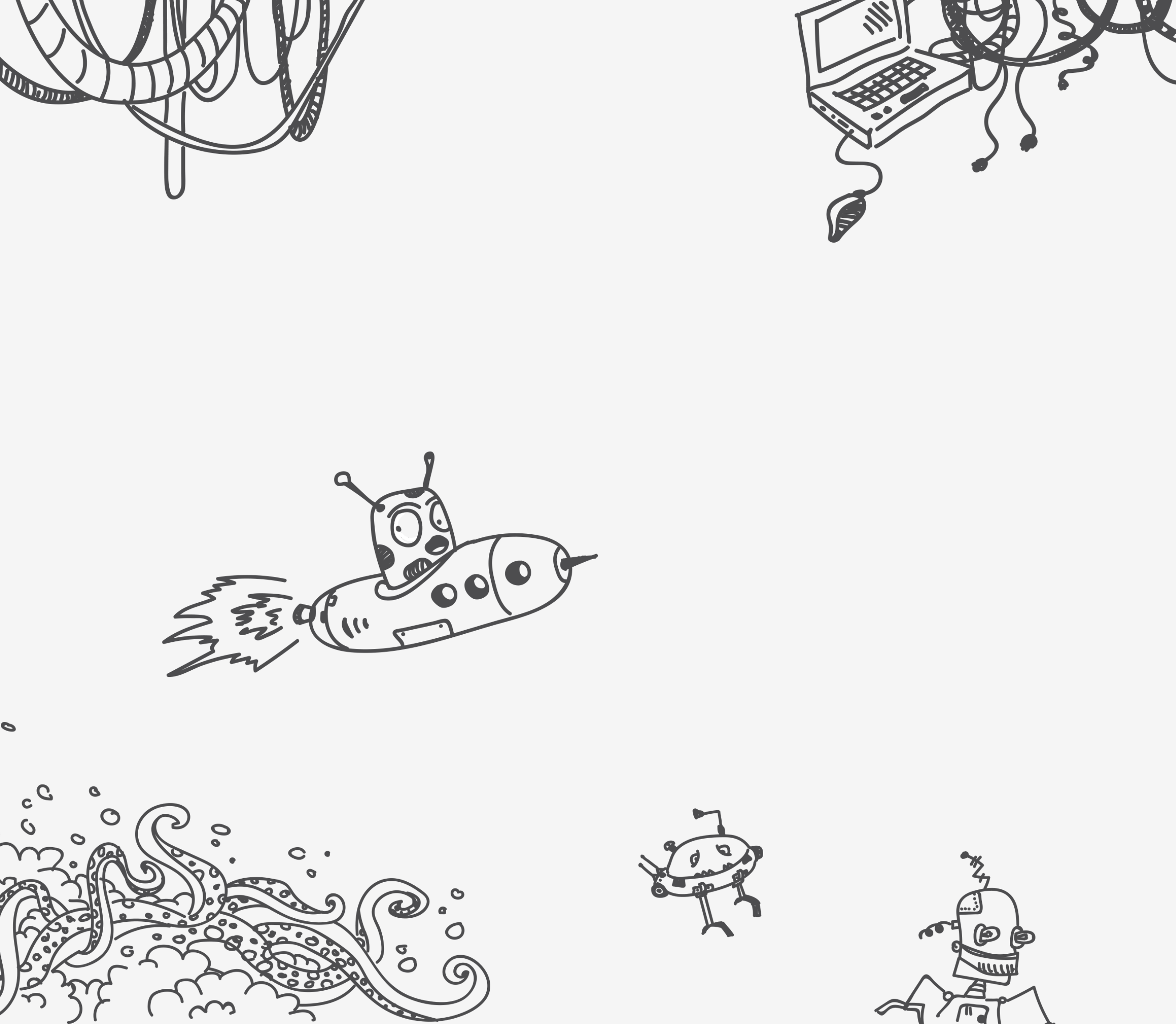
where:

- n - the number of observations in the data set,
- y_i - the actual value of the dependent variable for the i -th observation,
- $\hat{y}_i$ - the value predicted by the model for the i -th observation.
- $(\dots)^2$ - denotes the square of the error value
- $\sqrt{\dots}$ - denotes the square root of the expression in parentheses

Root Mean Squared Error

We can count the RMSE using the same method as the MSE. The only change is to define the **squared** argument with the value of *False*:

```
import numpy as np
from sklearn.metrics import mean_squared_error
np.sqrt(mean_squared_error(y_pred, y_true))
```



R^2 coefficient of determination

R² coefficient of determination

The coefficient of determination R² is a metric we have already encountered in this course. R² is a popular metric used to determine the accuracy of a model. The larger R² value indicates better model fit. R² takes values in the range 0-1, where 1 is an ideal model and 0 is a model whose results are no different from a completely random model.

The general formula for the coefficient of determination can be expressed by the formula:

$$R^2 = 1 - \frac{SSE}{SST}$$

where:

- **SSE** - is the sum of squares of the differences between the actual and predicted values,
- **SST** - is the sum of squares of the differences between the actual value and the average actual value

What R² shows can be interpreted as a measure that will determine how much better our model is relative to the base model (represented as the average actual value). An R² score of 0.85, for example, means that the model can handle 85% of the variance (dispersion) present in the data.

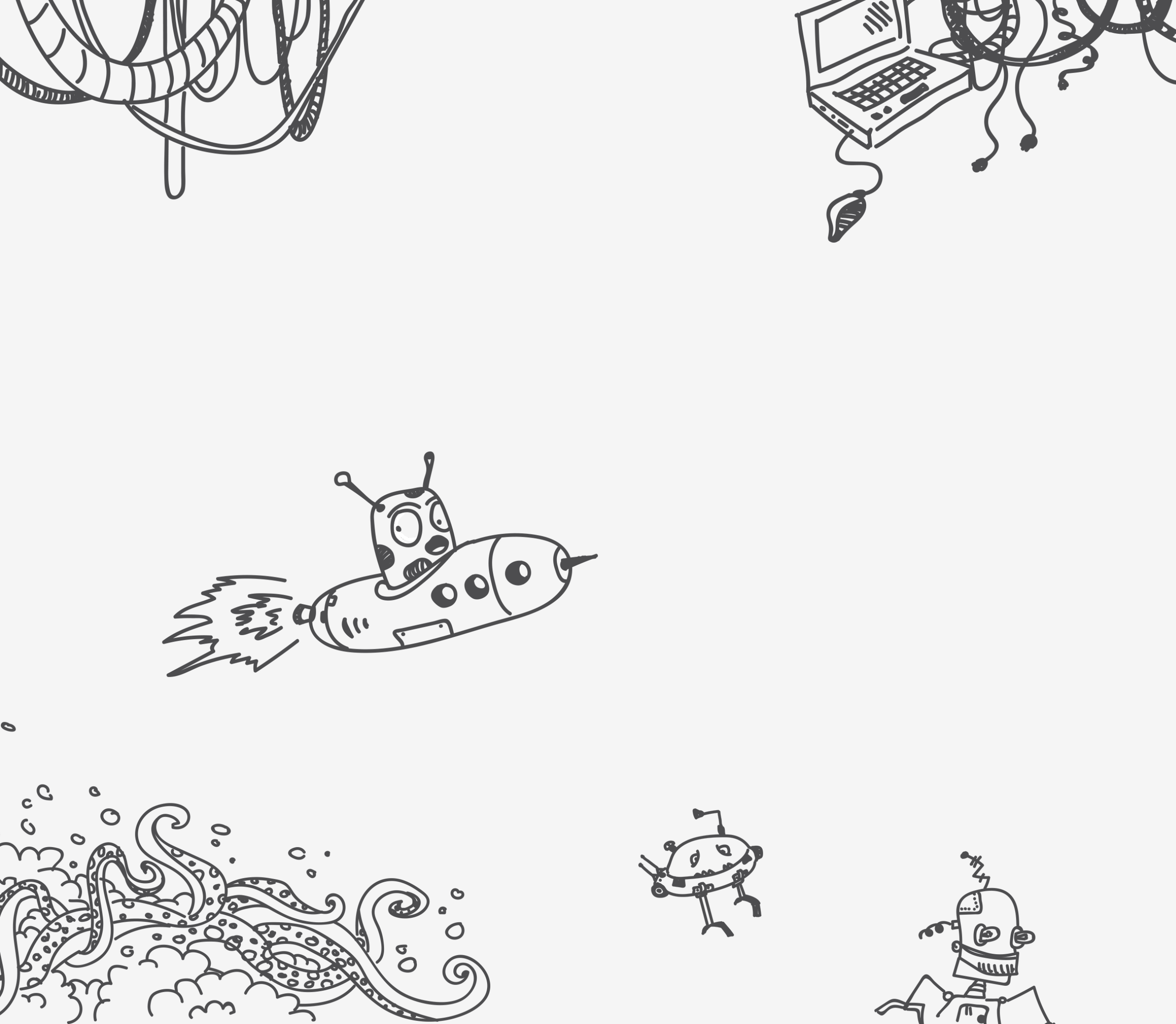
R² coefficient of determination

Let's assume that we have a dataset consisting of 6 variables (including 5 independent variables). Based on these, we built a regression model. However, it turns out that we have gained a new variable that we can use for the model. After training the model with the expanded data, it turns out that the R² has increased. This is because as we add more data, the variance of the data increases, resulting in the R² value increasing (or staying the same). This can be problematic if our variable is insignificant to our problem. Then R² may nevertheless grow, which may not fully reflect the actual accuracy of the model.

We can calculate the R² coefficient using the *r2_score method*. R² is also the default metric of a linear regression model, the result of which is returned when the *.score()* method is called on the model:

```
from sklearn.metrics import r2_score  
r2_score(y_pred, y_true)
```

R² loses its interpretability if it is calculated to evaluate *nonlinear* models.



Adjusted coefficient of determination R^2

Adjusted coefficient of determination R^2

Adjusted coefficient of determination R^2 - sounds a bit complicated, but in fact it is R^2 that solves the problem we described above. The disadvantage of the R^2 metric is that as new variables (features) are added, its score is constant or increases, but never decreases. This is because it assumes that more data = more variance.

The problem is that when we add an irrelevant feature to the model, sometimes R^2 increases, which is not a correct behavior.

The formula for the adjusted coefficient of determination R^2 is as follows:

$$R^2_{adj} = 1 - \frac{(1 - R^2)(n - 1)}{n - k - 1}$$

where:

- R^2 coefficient of determination
- **n** - the number of observations in the data set,
- **k** - the number of independent variables.

Adjusted coefficient of determination R^2

If previously our data consisted of a smaller number of features, for example, $k = 8$, then increasing k decreases the denominator, which leads to an increase in the value of the expression. By subtracting its value from 1, the total result of the adjusted R^2 will decrease relative to the classic R^2 .

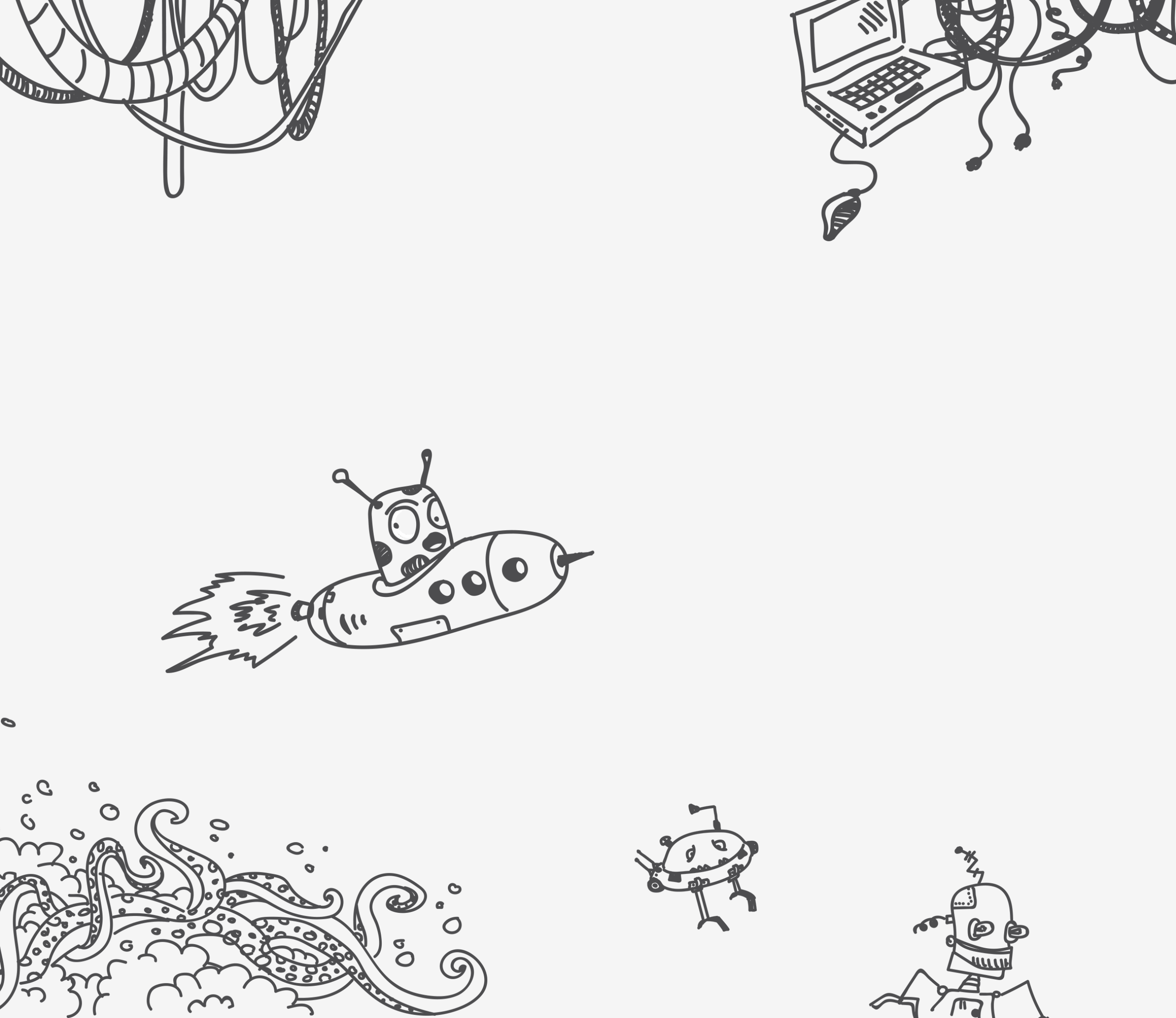
Let's consider two scenarios:

- The first scenario assumes the addition of an irrelevant variable to the training dataset. The R^2 score will remain constant or gently increase. Adjusted R^2 will decrease.
- The second scenario assumes the addition of a feature that significantly affects the outcome of our model. In this case, R^2 will increase, and the value of the expression: $1 - R^2$ will decrease. The decrease in the value of this expression, along with the decrease in the value of the denominator, causes the whole expression to increase. After subtracting it from 1, we get the increased value of the adjusted R^2 coefficient.

Adjusted coefficient of determination R^2

Using the `r2_score` method, we can also calculate the adjusted coefficient of determination:

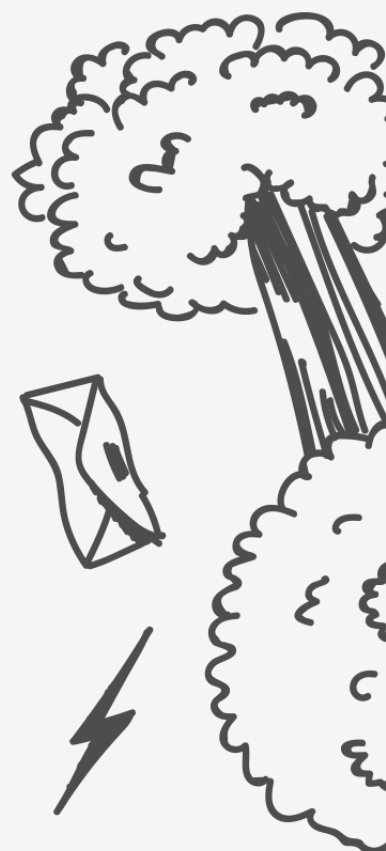
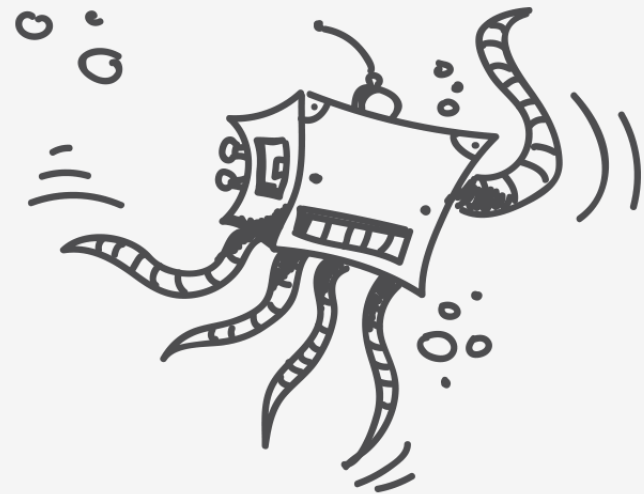
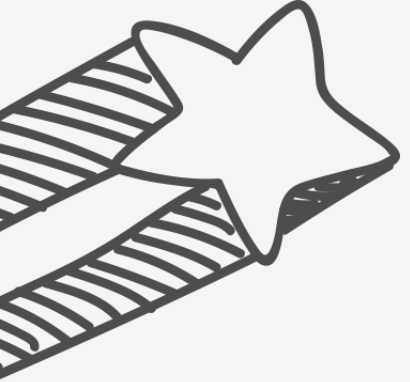
```
# R2 adjusted
def adjusted_r2_score(r2, n, k):
    adjusted_r2 = 1 - ((1 - r2) * (n - 1) / (n - k - 1))
    return adjusted_r2
```



Summary

Summary

In this presentation, we discussed 5 metrics that will be useful for the rest of our learning related to regression. You need to know that there is no single metric that always works correctly and helps you build a good model. What metrics you should use to evaluate a model largely depends on the characteristics of the problem. A good way to better understand what each metric means is to calculate its value. This will help you figure out which metric you can use when solving a given problem.



Practice time